

Lab 7 – Hashing

1. Mục tiêu

Kết thúc Lab 7, sinh viên có thể:

- Hiểu và code được rolling hash kiểu “cửa sổ trượt” cho bài toán tìm chuỗi con (Rabin–Karp mức cơ bản).
- So sánh trực tiếp với so khớp brute force về độ phức tạp ý tưởng.
- Áp dụng hash map / hash set để giải ba bài kinh điển:
 - Group anagrams (nhóm các từ là hoán vị của nhau).
 - Longest consecutive sequence (dãy liên tiếp dài nhất) trong $O(n)$.
 - Subarray sum = k bằng prefix sum + hash map.

2. Cấu trúc Lab

Bài 1 – Rolling hash & Rabin–Karp

File: lab7_bai1_rolling_hash.py

1.1. Bài toán

Cho:

- Chuỗi text T (ví dụ: "ababcbcababd").
- Chuỗi pattern P (ví dụ: "abc").

Yêu cầu:

- Viết 2 hàm:
 - `brute_force_search(text, pattern)` – trả về index đầu tiên P xuất hiện trong T (hoặc -1).

- `rolling_hash_search(text, pattern)` – dùng rolling hash đơn giản để tìm.
- So sánh kết quả hai hàm, in ra index tìm được.

1.2. Hàm brute-force

```
def brute_force_search(text, pattern):  
    """  
    So khớp trực tiếp: thử tất cả vị trí i, so sánh pattern với text[i : i+m].  
    Trả về index đầu tiên tìm được, hoặc -1 nếu không có.  
    """  
    n, m = len(text), len(pattern)  
    if m == 0 or m > n:  
        return -1  
  
    for i in range(n - m + 1):  
        # so m ký tự  
        if text[i:i+m] == pattern:  
            return i  
  
    return -1
```

1.3. Hàm hash đơn giản

```
def simple_hash(s):  
    """  
    Hash đơn giản: tổng mã ASCII các ký tự.  
    Dùng để minh họa rolling hash, không dùng cho bảo mật.  
    """  
    return sum(ord(c) for c in s)
```

1.4. Skeleton rolling hash search

```

def rolling_hash_search(text, pattern):
    """
    Tìm pattern trong text bằng rolling hash đơn giản.
    Sử dụng simple_hash + cửa sổ trượt.
    """
    n, m = len(text), len(pattern)
    if m == 0 or m > n:
        return -1

    # 1. Tính hash pattern
    hash_p = simple_hash(pattern)

    # 2. Tính hash cửa sổ đầu tiên của text
    window = text[:m]
    hash_w = simple_hash(window)

    # 3. Trượt cửa sổ
    for i in range(n - m + 1):
        # Nếu hash trùng → kiểm tra lại bằng so chuỗi
        if hash_w == hash_p:
            if text[i:i+m] == pattern:
                return i

        # Cập nhật hash_w cho cửa sổ tiếp theo
        if i < n - m:
            # TODO: cập nhật rolling hash:
            # - trừ ký tự text[i]
            # - cộng ký tự text[i + m]
            pass

    return -1

```

Việc của sinh viên:

- Điền phần cập nhật rolling hash:

```
hash_w -= ord(text[i])
hash_w += ord(text[i + m])
```

1.5. Hàm test rolling hash

```
def test_rolling_hash():
    text = "ababcabcababd"
    pattern = "abc"

    i1 = brute_force_search(text, pattern)
    i2 = rolling_hash_search(text, pattern)

    print("Text :", text)
    print("Pattern:", pattern)
    print("Brute force index :", i1)
    print("Rolling hash index:", i2)

if __name__ == "__main__":
    test_rolling_hash()
```

Yêu cầu:

- Đảm bảo hai hàm trả cùng kết quả.
- Có thể test nhiều pattern khác nhau.

Bài 2 – Group anagrams

File: lab7_bai2_group_anagrams.py

2.1. Bài toán

Cho mảng words gồm các string.

- Nhóm các từ là **anagram** của nhau.
- Anagram: cùng ký tự, khác thứ tự.
- Ví dụ:
 - Input: ["eat", "tea", "tan", "ate", "nat", "bat"]
 - Output (thứ tự nhóm không quan trọng):
 - [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]].

Yêu cầu:

- Cài `group_anagrams(words)` dùng hash map.

2.2. Gợi ý ý tưởng

Cách đơn giản: **sort từng từ** làm key:

- "eat", "tea", "ate" → "aet".
- "tan", "nat" → "ant".

Dùng dict:

- key: chuỗi đã sort.
- value: list các từ thuộc nhóm.

2.3. Skeleton code

```
from collections import defaultdict

def group_anagrams(words):
    """
    Nhóm các từ là anagram bằng hash map.
    Ý tưởng: dùng ".join(sorted(word))" làm key.
    """
    groups = defaultdict(list)
```

```

for w in words:
    # TODO:
    # 1. Tạo key bằng cách sort w
    # 2. Thêm w vào groups[key]
    pass

# Trả về list các group
return list(groups.values())

```

Phần cần điền:

```

key = ".join(sorted(w))
groups[key].append(w)

```

2.4. Hàm test group anagrams

```

def test_group_anagrams():
    words = ["eat", "tea", "tan", "ate", "nat", "bat"]
    result = group_anagrams(words)
    print("Input :", words)
    print("Output groups:")
    for group in result:
        print(" ", group)

if __name__ == "__main__":
    test_group_anagrams()

```

Sinh viên kiểm tra:

- Các từ anagram có nằm chung group hay không.

Bài 3 – Longest consecutive sequence

File: lab7_bai3_longest_consecutive.py

3.1. Bài toán

Cho mảng nums **không được sort trước**.

- Tìm độ dài **dãy số liên tiếp dài nhất**.
- Ví dụ:
 - [100, 4, 200, 1, 3, 2] → dãy [1,2,3,4] → 4.
 - [0,3,7,2,5,8,4,6,0,1] → dãy [0,1,2,3,4,5,6,7,8] → 9.

Yêu cầu: $O(n)$.

3.2. Ý tưởng hash set

Dùng set để kiểm tra tồn tại $O(1)$:

- B1: đưa toàn bộ nums vào set.
- B2: với mỗi x trong set, chỉ bắt đầu đếm khi x là **đầu dãy**: x-1 không nằm trong set.
- B3: từ x, tăng dần x+1, x+2,... cho đến khi không còn trong set, đếm độ dài.

Mỗi phần tử là đầu dãy nhiều nhất một lần → $O(n)$.

3.3. Skeleton code

```
def longest_consecutive(nums):  
    """  
    Trả về độ dài dãy liên tiếp dài nhất trong nums.  
    Dùng hash set để đạt  $O(n)$ .  
    """  
  
    if not nums:  
        return 0  
  
    num_set = set(nums)  
    best = 0
```

```

for x in num_set:
    # TODO:
    # chỉ bắt đầu nếu x là "đầu dãy" (x-1 không trong set)
    # sau đó đếm length từ x, x+1, x+2,...
    pass

return best

```

Phần cần điền (gợi ý):

```

if x - 1 not in num_set:
    length = 1
    cur = x
    while cur + 1 in num_set:
        cur += 1
        length += 1
    if length > best:
        best = length

```

3.4. Hàm test longest consecutive

```

def test_longest_consecutive():
    nums1 = [100, 4, 200, 1, 3, 2]
    nums2 = [0, 3, 7, 2, 5, 8, 4, 6, 0, 1]

    print("nums1:", nums1)
    print("Longest consecutive:", longest_consecutive(nums1)) # 4

    print("nums2:", nums2)
    print("Longest consecutive:", longest_consecutive(nums2)) # 9

if __name__ == "__main__":
    test_longest_consecutive()

```

Bài 4 – Subarray sum = k

File: lab7_bai4_subarray_sum_k.py

4.1. Bài toán

Cho:

- Mảng `nums` (có thể âm, dương).
- Số nguyên `k`.

Yêu cầu:

- Đếm số **subarray liên tiếp** có tổng bằng `k`.

Ví dụ:

- `nums = [1, 1], k=2` → có 2 subarray: (0..1), (1..2).
- `nums = [1, 1, 1], k=3` → subarray: [1, 1, 1], → kết quả 2.

Mục tiêu: $O(n)$.

4.2. Ý tưởng prefix sum + hash map

Gọi `prefix[i]` = tổng từ index 0..i.

Tổng subarray (l..r) = `prefix[r] - prefix[l-1]`.

Muốn: `prefix[r] - prefix[l-1] = k` → `prefix[l-1] = prefix[r] - k`.

Duyệt từ trái sang phải:

- `current_sum` = prefix tại vị trí hiện tại.
- Cần biết trước đó có bao nhiêu prefix = `current_sum - k`.
- Dùng dict `prefix_count` lưu: prefix value → số lần xuất hiện.
- Mỗi bước cập nhật `result += prefix_count[current_sum - k]`.

4.3. Skeleton code

```
from collections import defaultdict

def subarray_sum(nums, k):
    """
    Đếm số subarray liên tiếp có tổng = k.
    Dùng prefix sum + hash map.
    """
    prefix_count = defaultdict(int)
    prefix_count[0] = 1 # prefix sum = 0 xuất hiện 1 lần trước khi duyệt

    current_sum = 0
    result = 0

    for x in nums:
        current_sum += x

        # TODO:
        # 1. Tính cần_prefix = current_sum - k
        # 2. Nếu cần_prefix đã xuất hiện, cộng số lần vào result
        # 3. Cập nhật prefix_count[current_sum]
        pass

    return result
```

Phần cần điền:

```
needed = current_sum - k
if needed in prefix_count:
    result += prefix_count[needed]
prefix_count[current_sum] += 1
```

4.4. Hàm test subarray sum = k

```
def test_subarray_sum():
    nums1 = [1, 1, 1]
    k1 = 2
    print("nums1:", nums1, "k =", k1)
    print("Số subarray sum = k:", subarray_sum(nums1, k1)) # 2

    nums2 = [1, 2, 3]
    k2 = 3
    print("\nnums2:", nums2, "k =", k2)
    print("Số subarray sum = k:", subarray_sum(nums2, k2)) # 2

if __name__ == "__main__":
    test_subarray_sum()
```